

HW3(FEA): Appiah Francis

I(iii) This is the numerical results for (i), where $x_k = 0.5$ is a node.

Table 1: Errors and convergence orders for $k_1 = 1, k_2 = 5$

$N + 1$	$\ e_h\ _{L^2}$	Order	$\ e_h\ _{L^\infty}$	Order
10	6.5420E-04	—	1.1111E-03	—
20	1.6355E-04	2.00	2.7778E-04	2.00
40	4.0888E-05	2.00	6.9444E-05	2.00
80	1.0222E-05	2.00	1.7361E-05	2.00
160	2.5555E-06	2.00	4.3403E-06	2.00

Table 2: Errors and convergence orders for $k_1 = 3, k_2 = 10$

$N + 1$	$\ e_h\ _{L^2}$	Order	$\ e_h\ _{L^\infty}$	Order
10	2.2325E-04	—	3.7037E-04	—
20	5.5812E-05	2.00	9.2593E-05	2.00
40	1.3953E-05	2.00	2.3148E-05	2.00
80	3.4883E-06	2.00	5.7870E-06	2.00
160	8.7207E-07	2.00	1.4468E-06	2.00

Table 3: Errors and convergence orders for $k_1 = 5, k_2 = 9$

$N + 1$	$\ e_h\ _{L^2}$	Order	$\ e_h\ _{L^\infty}$	Order
10	1.4677E-04	—	2.2222E-04	—
20	3.6692E-05	2.00	5.5556E-05	2.00
40	9.1731E-06	2.00	1.3889E-05	2.00
80	2.2933E-06	2.00	3.4722E-06	2.00
160	5.7332E-07	2.00	8.6806E-07	2.00

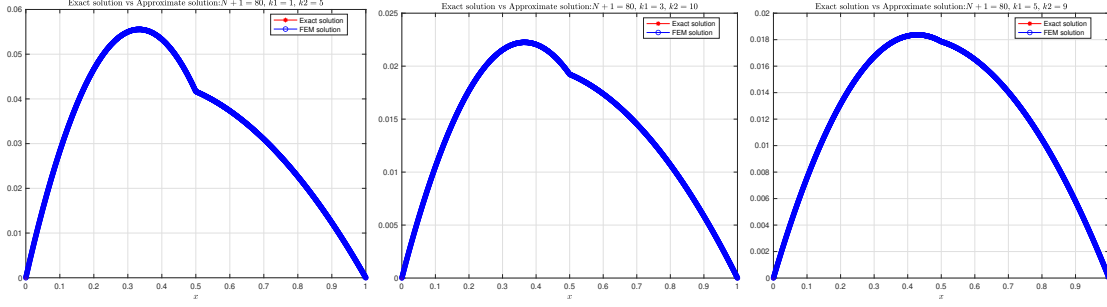


Figure 1: Graph of FEM and Exact solution When $x_k = 0.5$ is a node

This is the result for the variant if the interface is not aligned with the mesh

Table 4: Errors and convergence orders for $k_1 = 1$, $k_2 = 5$ (Variant)

$N + 1$	$\ e_h\ _{L^2}$	Order	$\ e_h\ _{L^\infty}$	Order
11	2.7733E-03	—	6.2895E-03	—
21	1.4381E-03	1.02	3.3510E-03	0.97
41	7.3268E-04	1.01	1.7326E-03	0.99
81	3.6985E-04	1.00	8.8137E-04	0.99
161	1.8582E-04	1.00	4.4456E-04	1.00

Table 5: Errors and convergence orders for $k_1 = 3$, $k_2 = 10$ (Variant)

$N + 1$	$\ e_h\ _{L^2}$	Order	$\ e_h\ _{L^\infty}$	Order
11	7.4602E-04	—	1.6745E-03	—
21	3.8222E-04	1.03	8.7720E-04	1.00
41	1.9373E-04	1.02	4.4959E-04	1.00
81	9.7570E-05	1.01	2.2768E-04	1.00
161	4.8969E-05	1.00	1.1458E-04	1.00

Table 6: Errors and convergence orders for $k_1 = 5$, $k_2 = 9$ (Varaint)

$N + 1$	$\ e_h\ _{L^2}$	Order	$\ e_h\ _{L^\infty}$	Order
11	2.6975E-04	—	5.8623E-04	—
21	1.2973E-04	1.13	2.8535E-04	1.11
41	6.4212E-05	1.05	1.4029E-04	1.06
81	3.2064E-05	1.02	6.9489E-05	1.03
161	1.6040E-05	1.01	3.4572E-05	1.02

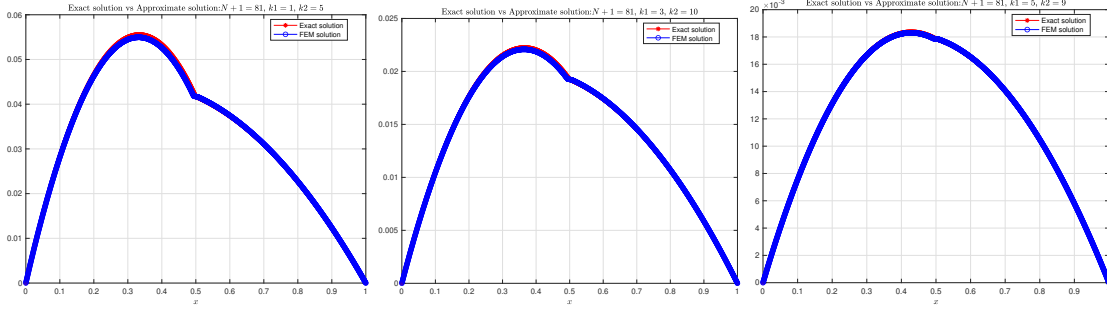
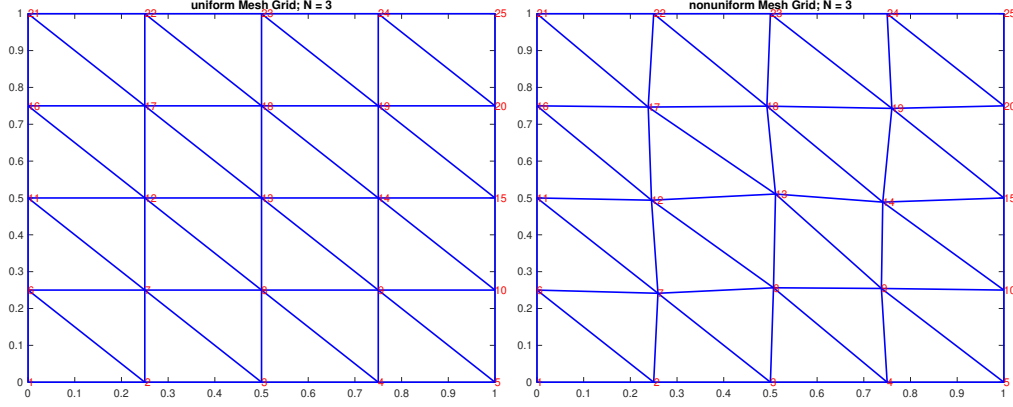


Figure 2: Graph of FEM and Exact solution When $x_k = 0.5$ does not align with the mesh

The numerical experiments clearly shows the effect of interface alignment on the accuracy and convergence of the finite element method for the given interface problem. When the interface location $x = 0.5$ is aligned with mesh nodes, the finite element solution convergence rates aligns with the theory. In particular, the L^2 and L^∞ error shows second-order convergence, This is expected, since the mesh exactly resolves the discontinuity in the derivative of the solution at a nodal point. In contrast, when the interface point $x = 0.5$ does not coincide with a mesh node, the convergence rates is approximately first order in both L^2 and L^∞ norms. In this case, the interface lies between an element. The element containing the interface becomes the dominant source of error, which limits the overall accuracy of the method. These results highlight the importance of mesh alignment in interface problems.

(a) Mesh Type used for the implementation



(b) The code is Implemented with a uniform mesh $h_j = h = \frac{1}{N+1}$, $p(x, y) = 3$, $q(x, y) = 2$. The exact solution is given by

$$u(x, y) = 1, x, y$$

with it's forcing respectively given as

$$f(x, y) = 2, 2x, 2y.$$

Table 7: L^2 Errors for uniform mesh

	$u(x, y) = 1$	$u(x, y) = x$	$u(x, y) = y$
$N + 1$	$\ e_h(\cdot)\ _{L^2(\Omega)}$	$\ e_h(\cdot)\ _{L^2(\Omega)}$	$\ e_h(\cdot)\ _{L^2(\Omega)}$
20	3.8773E-15	2.3988E-15	2.2512E-15
40	2.4440E-14	1.2744E-14	1.2961E-14

Table 8: L^2 Errors for nonuniform mesh

	$u(x, y) = 1$	$u(x, y) = x$	$u(x, y) = y$
$N + 1$	$\ e_h(\cdot)\ _{L^2(\Omega)}$	$\ e_h(\cdot)\ _{L^2(\Omega)}$	$\ e_h(\cdot)\ _{L^2(\Omega)}$
20	6.716E-16	2.991E-16	6.021E-16
40	3.259E-15	1.927E-15	8.246E-16

(c) For exact solution, $u(x, y) = y^3 + \sin(5(x + y)) + 2e^x$. The partial derivatives are given below

$$\begin{aligned} u_x(x, y) &= 5 \cos(5(x + y)) + 2e^x, \\ u_{xx}(x, y) &= -25 \sin(5(x + y)) + 2e^x, \\ u_y(x, y) &= 3y^2 + 5 \cos(5(x + y)), \\ u_{yy}(x, y) &= 6y - 25 \sin(5(x + y)). \end{aligned}$$

Table 9: Errors, convergence orders, and computational time for uniform mesh

$N + 1$	$\ e_h\ _{L^2}$	Order	$\ e_h\ _{H^1}$	Order	Assembly Time (s)	Solve Time (s)
10	2.9578E-02	–	7.4805E-01	–	8.8325E-03	1.4913E-04
20	7.5020E-03	1.98	3.7355E-01	1.00	9.4547E-03	2.6960E-03
40	1.8824E-03	1.99	1.8670E-01	1.00	3.1137E-02	2.3372E-02
80	4.7104E-04	2.00	9.3340E-02	1.00	1.2474E-01	4.7815E-01
160	1.1779E-04	2.00	4.6669E-02	1.00	5.3350E-01	3.0517E+01

Table 10: Errors, convergence orders, and computational time for non-uniform mesh

$N + 1$	$\ e_h\ _{L^2}$	Order	$\ e_h\ _{H^1}$	Order	Assembly Time (s)	Solve Time (s)
10	2.956E-02	–	7.481E-01	–	8.130E-03	1.405E-04
20	7.581E-03	1.96	3.751E-01	1.00	8.597E-03	2.274E-03
40	1.902E-03	2.00	1.875E-01	1.00	2.758E-02	1.849E-02
80	4.765E-04	2.00	9.380E-02	1.00	1.110E-01	5.115E-01
160	1.191E-04	2.00	4.689E-02	1.00	4.884E-01	3.930E+01

1 Conclusion

The numerical results demonstrate that the finite element method achieves the expected optimal convergence rates for linear elements ($K \in P^1$). In particular, the L^2 error converges at approximately second order, $\mathcal{O}(h^2)$, while the H^1 error converges at first order, $\mathcal{O}(h)$. These rates are consistent with the theory for piecewise linear finite element approximations. A comparison between uniform and non-uniform meshes shows that both have the same convergence behavior. The non-uniform mesh does not affect the accuracy of the solution, and the convergence orders remain optimal in both norms. The errors decrease smoothly as the mesh is refined. The behavior of the solution aligns well with theoretical expectations. For computation, the matrix assembly time increases moderately with the number of elements, which is consistent with the expected linear scaling of element-wise assembly procedures. In contrast, the time required to solve the linear system grows rapidly as the problem size increases. This is due to the use of a direct solver, which becomes increasingly expensive for larger systems even when the matrix is sparse. As a result, the dominant computational cost shifts from assembly to the solution phase as the mesh is refined. The primary limitation observed is computational efficiency at larger problem sizes, which could be improved by employing iterative solvers that better exploit the sparsity of the system matrix.

Listing 1: main.m

```
1  clc;
2  clear;
3  close all;
4
5  %% Parameters
6  Nvalue = [9,19,39,79,159];
7  %Nvalue = 3;
8  par.xa = 0;
9  par.xb = 1;
10 par.ya = 0;
11 par.yb = 1;
12 par.p = 3;
13 par.q = 2;
14
15 %par.meshType = "uniform"; % or "non_uniform"
16 par.meshType = "nonuniform";
17 par.testfunc = 4;          % use 4 for meaningful convergence
18
19 %% Storage
20 l2norm = zeros(length(Nvalue),1);
21 enorm = zeros(length(Nvalue),1);
22 AssembleTime = zeros(length(Nvalue),1);
23 SolveMatTime = zeros(length(Nvalue),1);
24 %% Solve for each mesh
25 for j = 1:length(Nvalue)
26     par.N = Nvalue(j);
27     [errL2, errH1, MatAssemTime, solveMat] = twoPointBVP2D(par);
28     l2norm(j) = errL2;
29     enorm(j) = errH1;
30     AssembleTime(j) = MatAssemTime;
31     SolveMatTime(j) = solveMat;
32 end
33
```

```

34 %% Compute mesh size h
35 h = 1./(Nvalue(:)-1);
36
37 %% Compute convergence rates
38 l2conv = NaN(length(Nvalue),1);
39 Econv = NaN(length(Nvalue),1);
40
41 for j = 1:length(Nvalue)-1
42     l2conv(j+1) = log(l2norm(j)/l2norm(j+1)) / log(2);
43     Econv(j+1) = log(enorm(j)/enorm(j+1)) / log(2);
44 end
45
46 %% Create results table
47 Results = table(Nvalue(:), h, l2norm, l2conv, enorm, Econv, ...
48     AssembleTime,SolveMatTime,'VariableNames',...
49     {'N','h','L2_Error','L2_Order','H1_Error','H1_Order','cpuTime_MatAss(s)','cpuTime_solvMat(s)'});
50
51 %% Display results
52 format short e
53 disp('=====')
54 disp(['Mesh_Type: ', char(par.meshType)])
55 disp(['Test_Function: ', num2str(par.testfunc)])
56 disp('=====')
57 disp(Results)
58
59 %% Optional: save to file
60 writetable(Results, 'FEM_results.csv');
61
62 %% Optional: log-log plot
63 % figure
64 % loglog(h, l2norm, '-o', 'LineWidth',2)
65 % hold on
66 % loglog(h, enorm, '-s', 'LineWidth',2)
67 % grid on
68 % xlabel('h')
69 % ylabel('Error')
70 % legend('L2 Error','H1 Error','Location','southEast')
71 % title('FEM Convergence')

```

Listing 2: Mesh generator.m

```

1 function[P,T] = Meshgen(par)
2 N = par.N;
3 xa = par.xa;
4 ya = par.ya;
5 meshType = par.meshType;
6 Nxtot = N + 2;
7 Nytot = N + 2;
8 Ntot = Nytot*Nxtot;
9 Nele = 2*(N+1)*(N+1);
10 P = zeros(Ntot,2);
11 T = zeros(Nele,3);
12 x = zeros(Nxtot,1);
13 y = zeros(Nytot,1);
14 x(1) = xa;
15 y(1) = ya;
16 ind = @(i,j) (j-1)*Nxtot + i;
17 h = 1/(N+1);

```

```

18 for j = 1 : N+1
19     x(j+1) = h + x(j);
20     y(j+1) = h + y(j);
21 end
22
23 for j = 1 : Nxtot
24     for i = 1:Nytot
25         ind_glb = ind(i,j);
26         P(ind_glb,1) = x(i);
27         P(ind_glb,2) = y(j);
28     end
29 end
30 %% perturbed
31 if strcmp(meshType,'nonuniform')
32     tol = 1e-12;
33     for k = 1:size(P,1)
34
35         x = P(k,1);
36         y = P(k,2);
37
38         % check if interior node
39         if (x > tol && x < 1 - tol && y > tol && y < 1 - tol)
40
41             Rx = rand - 0.5;
42             Ry = rand - 0.5;
43
44             P(k,1) = P(k,1) + (h/10)*Rx;
45             P(k,2) = P(k,2) + (h/10)*Ry;
46
47         end
48     end
49 end
50
51 % elements
52 e = 1;
53 for j = 1:N + 1
54     for i = 1:N + 1
55
56         n1 = (j-1)*(Nxtot) + i;
57         n2 = n1 + 1;
58         n3 = n1 + Nxtot;
59         n4 = n3 + 1;
60
61         T(e,:) = [n1 n2 n3];
62         T(e+1,:) = [n2 n4 n3];
63
64         e = e+2;
65
66     end
67 end
68 if par.N < 5
69     triplot(T, P(:,1), P(:,2),'Linewidth',1.5)
70     %axis equal
71     hold on
72
73     for i = 1:size(P,1)
74         text(P(i,1), P(i,2), sprintf('%d', i), 'Color','r')
75     end
76 hold off

```

```

77 title(sprintf('%s_Mesh_Grid;_N_=%d',meshType, N))
78 filename = sprintf('img/mesh_%s.eps',meshType);
79 print('-depsc2', filename);
80 end
81 return
82 end

```

Listing 3: twoPointBVP.m

```

1 function [errL2,errH1,MatAssemTime,solveMat] = twoPointBVP2D(par)
2
3 % define functions
4 if par.testfunc == 1
5     fun.UTRUE = @(x,y) 1 + 0*x;
6     fun.UTRUEX = @(x,y) 0;
7     fun.UTRUEXX = @(x,y) 0;
8     fun.UTRUEY = @(x,y) 0;
9     fun.UTRUEYY = @(x,y) 0;
10 elseif par.testfunc == 2
11     fun.UTRUE = @(x,y) x;
12     fun.UTRUEX = @(x,y) 1 + 0*x;
13     fun.UTRUEXX = @(x,y) 0;
14     fun.UTRUEY = @(x,y) 0;
15     fun.UTRUEYY = @(x,y) 0;
16 elseif par.testfunc == 3
17     fun.UTRUE = @(x,y) y;
18     fun.UTRUEX = @(x,y) 0;
19     fun.UTRUEXX = @(x,y) 0;
20     fun.UTRUEY = @(x,y) 1 + 0*y;
21     fun.UTRUEYY = @(x,y) 0;
22 else
23     fun.UTRUE = @(x,y) y.^3 + sin(5*(x+y)) + 2*exp(x);
24     fun.UTRUEX = @(x,y) 5*cos(5*(x+y)) + 2*exp(x);
25     fun.UTRUEXX = @(x,y) -25*sin(5*(x+y)) + 2*exp(x);
26     fun.UTRUEY = @(x,y) 3*y.^2 + 5*cos(5*(x+y));
27     fun.UTRUEYY = @(x,y) 6*y -25*sin(5*(x+y));
28 end
29 fun.FORCE = @(x,y) -par.p*(fun.UTRUEXX(x,y) + fun.UTRUEYY(x,y))...
30     + par.q * (fun.UTRUE(x,y));
31
32 % mesh generation
33 [P, T] = Meshgen(par);
34 x = P(:,1);
35 y = P(:,2);
36 %% boundary and interior points
37 tol = 1e-12;
38 boundary_nodes = find(abs(P(:,1)) < tol | abs(P(:,1)-1) < tol | ...
39     abs(P(:,2)) < tol | abs(P(:,2)-1) < tol );
40 interior_nodes = setdiff((1:size(P,1))',boundary_nodes);
41
42 %% form matrix and load vector
43 [A,F,Area,MatAssemTime] = formMatrixGlb(P,T,par,fun);
44
45 % Reduce matrix and load vector
46 AI = A(interior_nodes,interior_nodes);
47 B = A(interior_nodes,boundary_nodes);
48 Cb = fun.UTRUE(x(boundary_nodes),y(boundary_nodes));
49 FI = F(interior_nodes) - B * Cb;
50 tic

```

```

51 CI = AI\FI;
52 solveMat = toc;
53
54 %% solve for c
55 C = zeros(size(P,1),1);
56 C(boundary_nodes) = Cb;
57 C(interior_nodes) = CI;
58
59 %% compute error
60 [errL2,errH1] = getErrors(P,T,C,fun,Area);
61 %quadrature points
62 return
63 end

```

Listing 4: formMatrix.m

```

1  %ind : interior index
2  %P : vector of nodes
3  %T : matrix of elements
4  %area : vector of areas;
5  function [Ak,loadVecEle,K] = formMatrixEle(Pe,p,q,fun)
6  x1 = Pe(1,1); y1 = Pe(1,2);
7  x2 = Pe(2,1); y2 = Pe(2,2);
8  x3 = Pe(3,1); y3 = Pe(3,2);
9
10 K = 0.5 * abs((x2-x1)*(y3-y1)-(y2-y1)*(x3-x1));
11
12 % b(x coordinate for grad of lambda) and c (y coordinate) coefficients
13 b = [y2 - y3; y3 - y1; y1 - y2];
14 c = [x3 - x2; x1 - x3; x2 - x1];
15 %% Matrix components
16 Sk = zeros(3,3);
17 Mk = K/12 * [2 1 1; 1 2 1; 1 1 2];
18 for i = 1:3
19     for j = 1:3
20         Sk(i,j) = (b(i)*b(j) + c(i)*c(j)) / (4*K);
21     end
22 end
23 Ak = q*Mk + p*Sk;
24 fe = [fun.FORCE(x1,y1);fun.FORCE(x2,y2);fun.FORCE(x3,y3)];
25 loadVecEle = Mk * fe;
26 return
27 end

```

Listing 5: $U_h(x, y)$ function

```

1  function uh = UhElement(Pe,Ue,x,area)
2  lambda = zeros(3,1);
3  x1 = Pe(1,1); y1 = Pe(1,2);
4  x2 = Pe(2,1); y2 = Pe(2,2);
5  x3 = Pe(3,1); y3 = Pe(3,2);
6  xp = x(1);
7  yp = x(2);
8
9  % area
10 %area = 0.5 * abs(det([x2-x1, x3-x1; y2-y1, y3-y1]));
11 % barycentric coordinates
12 lambda(1) = ((x2 - xp)*(y3 - yp) - (x3 - xp)*(y2 - yp)) / (2*area);
13 lambda(2) = ((xp - x1)*(y3 - y1) - (x3 - x1)*(yp - y1)) / (2*area);

```

```

14 lambda(3) = ((x2 - x1)*(yp - y1) - (xp - x1)*(y2 - y1)) / (2*area);
15
16 uh = Ue' * lambda;
17 return
18 end

```

Listing 6: $e_h(x, y)$ function

```

1 function [glb_errL2 ,glb_errH1] = getErrors(P,T,C,fun,Area)
2 errL2 = 0;
3 errH1 = 0;
4 for e = 1:size(T,1)
5     nodes = T(e,:);
6     Pe = P(nodes,:);
7     Ce = C(nodes);
8
9     x1 = Pe(1,1); y1 = Pe(1,2);
10    x2 = Pe(2,1); y2 = Pe(2,2);
11    x3 = Pe(3,1); y3 = Pe(3,2);
12    area = Area(e);
13
14    %quadrature points
15    a1 = [x1,y1]; a2 = [x2,y2]; a3 = [x3,y3];
16    a12 = (a1+a2)/2; a13 = (a1+a3)/2; a23 = (a2 + a3)/2;
17    a123 = (a1 + a2 + a3)/3;
18
19    quadpts = [a1;a2;a3;a12;a13;a23;a123];
20    quadwts = [3;3;3;8;8;8;27];
21
22    grad_phi = 1/(2*area)*[y2-y3 x3-x2;y3-y1 x1-x3;y1-y2 x2-x1];
23    grad_uh = Ce' * grad_phi;
24
25    Ele_errL2 = 0;
26    Ele_errH1 = 0;
27    for q = 1:7
28        xq = quadpts(q,1);
29        yq = quadpts(q,2);
30
31        u = fun.UTRUE(xq,yq);
32        ux = fun.UTRUEX(xq,yq);
33        uy = fun.UTRUEY(xq,yq);
34        uh = UhElement(Pe,Ce,[xq,yq],area);
35
36        Ele_errL2 = Ele_errL2 + quadwts(q) * (uh - u)^2;
37        Ele_errH1 = Ele_errH1 + quadwts(q) ...
38            * ((grad_uh(1)-ux)^2 + (grad_uh(2)-uy)^2);
39    end
40    errL2 = errL2 + area/60 * Ele_errL2;
41    errH1 = errH1 + area/60 * Ele_errH1;
42 end
43 glb_errL2 = sqrt(errL2);
44 glb_errH1 = sqrt(errH1 + errL2);
45 return
46 end

```

Listing 7: main.m

```

1 paramSets = [
2     1, 5;

```

```

3     3, 10;
4     5, 9
5     ];
6     par.a = 0; par.b = 1;
7     par.testOption = 1;
8     par.debug = 0;
9     %Nvalue = [9,19,39,79,159];
10    Nvalue = [10,20,40,80,160];
11    results = [];
12
13    for p = 1:size(paramSets,1)
14
15        par.k1 = paramSets(p,1);
16        par.k2 = paramSets(p,2);
17
18        l2norm = zeros(length(Nvalue),1);
19        linfnorm = zeros(length(Nvalue),1);
20        h = zeros(length(Nvalue),1);
21
22
23        for j = 1:length(Nvalue)
24            par.N = Nvalue(j);
25
26            gpar = twoPointBVP(par);
27
28            h(j) = 1/(Nvalue(j)+1);
29            l2norm(j) = gpar.L2Norm;
30            linfnorm(j) = gpar.LinfNorm;
31            %enorm(j) = gpar.ENorm;
32
33            if Nvalue(j) == 80
34                figure(1);
35                plot(gpar.xq, gpar.u_vec,'r-*','LineWidth', 1)
36                hold on
37                plot(gpar.xq, gpar.uh_vec,'b-o','LineWidth', 1)
38                hold off
39                grid on;
40                xlim([0 1]);
41                legend('Exact_solution', 'FEM_solution', 'Location','best')
42                xlabel('$x$', 'Interpreter', 'latex');
43                title(sprintf('Exact_solution_vs_Approximate_solution:$N+1=%d$, $k1=%g$, $k2=%g$',
44                    Nvalue(j)+1,par.k1,par.k2), ...
45                    'Interpreter','latex')
46                filename = sprintf('img/bsolplot_N%d_k1%g_k2%g.eps',Nvalue(j)+1,par.k1,par.k2);
47                print('-depsc2', filename);
48            end
49
50            % convergence rates
51            l2conv = NaN(length(Nvalue),1);
52            linfconv = NaN(length(Nvalue),1);
53            Econv = NaN(length(Nvalue),1);
54
55            for j = 2:length(Nvalue)
56                l2conv(j) = log(l2norm(j-1)/l2norm(j)) ...
57                    / log((Nvalue(j)+1)/(Nvalue(j-1)+1));
58                linfconv(j) = log(linfnorm(j-1)/linfnorm(j))...
59                    / log((Nvalue(j)+1)/(Nvalue(j-1)+1));
60                % l2conv(j) = log(l2norm(j-1)/l2norm(j))/log(2);

```

```

61         % linfconv(j) = log(linfnorm(j-1)/linfnorm(j))/log(2);
62         % Econv(j)    = log(enorm(j-1)/enorm(j))/log(2);
63     end
64
65     % store EVERYTHING (errors + rates)
66     for j = 1:length(Nvalue)
67         results = [results;
68                 par.k1, par.k2, Nvalue(j), ...
69                 l2norm(j), linfnorm(j), ...
70                 l2conv(j), linfconv(j)];
71     end
72 end
73
74 T = array2table(results, ...
75     'VariableNames', {'k1','k2','N', ...
76     'L2Error','LinfError', ...
77     'L2Conv','LinfConv'});
78
79 disp(T)

```

Listing 8: twoPointBVP.m

```

1 function gpar = twoPointBVP(par)
2 a = par.a; b = par.b;
3 N = par.N; debug = par.debug;
4 h = 1/(N + 1);
5 k1 = par.k1;
6 k2 = par.k2;
7 x = zeros(N+2,1);
8 c = zeros(N+2,1);
9 x(1) = a;
10 for j = 1 : N+1
11     x(j+1) = h + x(j);
12 end
13 alpha = 0;
14 beta = 0;
15
16 f = @(x) 1 + 0*x;
17
18 A1 = (1/(2*k2) + 1/(8*k1) - 1/(8*k2)) / (0.5*(1 + k1/k2));
19 A2 = (k1/k2) * A1;
20 B2 = -1/(8*k1) + A1/2 + 1/(8*k2) - (k1/(2*k2))*A1;
21
22 UTRUE = @(x) ...
23     (-1/(2*k1) * x.^2 + A1*x) .* (x < 0.5) + ...
24     (-1/(2*k2) * x.^2 + A2*x + B2) .* (x >= 0.5);
25
26 UTRUEX = @(x) ...
27     (-1/(k1) * x.^2 + A1) .* (x < 0.5) + ...
28     (-1/(k2) * x.^2 + A2) .* (x >= 0.5);
29
30 % form matrix A
31 A = formMatrix(x,par);
32 %form vector F
33 F = zeros(N,1);
34 for i = 1:N
35     F(i) = f(x(i+2))*h/6 + f(x(i+1))*(2*h)/3 + f(x(i))*h/6;
36 end
37 % solve for c

```

```

38 c(1) = alpha;
39 c(N+2) = beta;
40 c(2:N+1) = A\F;
41
42 u = zeros(length(x),1);
43 for i = 1 : length(x)
44     u(i) = UTRUE(x(i));
45 end
46 %% calculate errors norms
47 M = 3;
48 L2Norm = 0;
49 LinfNorm = 0;
50 %ENorm = 0;
51 for j = 1: N+1
52     for m = 1:M
53         idx = x(j) + h*m/M;
54         e_h = UhFunc(idx,j,x,h,c) - UTRUE(idx);
55         %e_hx = UhFuncx(j,h,c) - UTRUEX(idx);
56         L2Norm = L2Norm + e_h^2* h/M;
57         LinfNorm = max(LinfNorm ,abs(e_h));
58         %ENorm = ENorm + e_hx^2* h/M;
59     end
60 end
61 gpar.L2Norm = sqrt(L2Norm);
62 %gpar.ENorm = sqrt(ENorm);
63 gpar.LinfNorm = LinfNorm;
64 %% error function
65 Mq = 20;
66 xq_vec = x(1);
67 % store FEM and exact solutions
68 uh_vec = UhFunc(x(1),1,x,h,c);
69 u_vec = UTRUE(x(1));
70 eh_vec = u_vec -uh_vec;
71 for j = 1:N+1
72     for m = 1:Mq
73         idxq = x(j) + h*m/Mq;
74         % compute solutions
75         uh = UhFunc(idxq,j,x,h,c);
76         u = UTRUE(idxq);
77         % error
78         e_h = uh - u;
79         % store values
80         xq_vec = [xq_vec; idxq];
81         eh_vec = [eh_vec; e_h];
82         uh_vec = [uh_vec; uh];
83         u_vec = [u_vec; u];
84     end
85 end
86 gpar.xq = xq_vec;
87 gpar.eh_vec = eh_vec;
88 gpar.u_vec = u_vec;
89 gpar.uh_vec = uh_vec;
90 %%
91 if debug == 1
92     % for testing uh function at the nodes
93     uh = zeros(length(x),1);
94     uexact = zeros(length(x),1);
95     for j = 1:N+1
96         uh(j) = UhFunc(x(j),j,x,h,c);

```

```

97     uexact(j) = UTRUE(x(j));
98 end
99 uh(N+2) = UhFunc(x(N+2),N+1,x,h,c);
100 uexact(N+2) = UTRUE(x(N+2));
101 figure(2)
102 plot(x, uh,'b-*','LineWidth', 1.5);
103 hold on
104 plot(x, uexact,'r-','LineWidth', 1.5);
105 hold on
106 plot(x, c,'k--','LineWidth', 1.5);
107 hold off;
108 grid on;
109 end
110 return
111 end

```

Listing 9: formMatrix.m

```

1  % function A = formMatrix(h,par,indk)
2  % % Define Matrix A
3  % k1 = par.k1;
4  % k2 = par.k2;
5  % N = par.N;
6  % A = zeros(N,N);
7  % % First row
8  % A(1,1) = k1*(1/h + 1/h);
9  % A(1,2) = -k1/h;
10 %
11 % % Interior rows
12 % for i = 2:N-1
13 %     if ( i < indk)
14 %         A(i,i-1) = -k1/h;
15 %         A(i,i) = k1*(1/h + 1/h);
16 %         A(i,i+1) = -k1/h;
17 %     elseif i == indk
18 %         A(i,i-1) = -k1/h;
19 %         A(i,i) = (k1/h + k2/h);
20 %         A(i,i+1) = -k2/h;
21 %     elseif (i > indk)
22 %         A(i,i-1) = -k2/h;
23 %         A(i,i) = k2*(1/h + 1/h);
24 %         A(i,i+1) = -k2/h;
25 %     end
26 % end
27 %
28 % % Last row
29 % A(N,N-1) = -k2/h;
30 % A(N,N) = k2*(1/h + 1/h);
31 % A = sparse(A);
32 % return
33 % end
34
35 function A = formMatrix(x,par)
36
37 k1 = par.k1;
38 k2 = par.k2;
39 N = par.N;
40
41 A = zeros(N,N);

```

```

42 tol = 1e-12;
43
44 for e = 1:N+1
45
46     xL = x(e);
47     xR = x(e+1);
48     h = xR - xL;
49
50     % determine coefficient on element
51     if xR <= 0.5 + tol
52         % fully in left region
53         coeff = k1 / h;
54
55     elseif xL >= 0.5 - tol
56         % fully in right region
57         coeff = k2 / h;
58
59     else
60         % --- interface cuts this element ---
61
62         h1 = 0.5 - xL;
63         h2 = xR - 0.5;
64
65         coeff = k1/h1 + k2/h2;
66
67     end
68
69     % local stiffness matrix
70     Ae = coeff * [1 -1; -1 1];
71
72     % map to global indices (interior only)
73     nodes = [e-1, e]; % interior indexing
74
75     for a = 1:2
76         for b = 1:2
77             i = nodes(a);
78             j = nodes(b);
79
80             %interior points
81             if i >= 1 && i <= N && j >= 1 && j <= N
82                 A(i,j) = A(i,j) + Ae(a,b);
83             end
84         end
85     end
86 end
87
88 A = sparse(A);
89 end

```

Listing 10: $U_h(x)$ function

```

1 function U_h = UhFunc(a,j,x,h,c)
2 U_h = (c(j)*(x(j+1)-a)+c(j+1)*(a-x(j)))/h;
3 return
4 end

```

Listing 11: $U_{hx}(x)$ function

```

1 function Uh_x = UhFuncx(j,h,c)
2 Uh_x = (-c(j)+c(j+1))/h;
3 return
4 end

```

2 Setup Description

In this section, we describe the main components of the reduced basis (RB) method used for the parametrized problem.

2.1 Spatial Discretization

We consider the domain $[0, 1]$ and discretize it using a uniform mesh with $N = 39$ interior nodes. The mesh is chosen such that the point $x = 0.5$ is included as a grid node, which is important due to the discontinuity in the reaction term.

2.2 Parameter Domain

The parameter vector is given by $\mu = (\mu_1, \mu_2) \in [0, 1] \times [0, 2]$.

2.3 Training and Test Sets

The training set D_{train} is constructed as a uniform tensor grid over the parameter domain with 30×30 samples, resulting in a total of 900 parameter points. This set is used in the greedy algorithm to construct the reduced basis.

The test set D_{test} consists of 100 randomly sampled parameters uniformly distributed over the parameter domain. It is used to evaluate the accuracy of the reduced model.

2.4 Reduced Basis Construction

The reduced basis is constructed using a greedy algorithm. Starting from an initial parameter value, the algorithm iteratively selects new parameters that maximize a chosen error indicator over the training set. The corresponding high-fidelity solutions are added to the basis.

Two variants of the basis are considered:

- A direct basis formed by the collection of snapshots,
- An orthonormal basis obtained via QR decomposition.

2.5 Error Estimation

Two types of error indicators are used:

- The ℓ^1 -norm of the reduced coefficients:

$$\Delta(\mu) = \|c(\mu)\|_1,$$

- A residual-based estimator:

$$\Delta(\mu) = \|f - A(\mu)u_m(\mu)\|.$$

The residual-based estimator provides a more reliable measure of the approximation error.

2.6 Stopping Criterion

The greedy algorithm is terminated when the maximum error indicator over the training set satisfies

$$\max_{\mu \in D_{\text{train}}} \Delta(\mu) < 10^{-6},$$

or when a maximum reduced basis size of $r_{\text{max}} = 15$ is reached.

2.7 Affine Decomposition and Precomputation

The bilinear form admits an affine decomposition of the form

$$A(\mu) = A_0 + \mu_1 A_1 + \mu_2 A_2.$$

This structure enables an efficient offline-online decomposition. The matrices A_0 , A_1 , and A_2 are assembled during the offline stage, while reduced systems are solved efficiently during the online stage.